

Summary

This video explains the principles and methodologies involved in constructing Minimal Deterministic Finite Automata (mDFA) for finite languages. A DFA is a complete computational model that must accept all valid strings in a language and explicitly reject all invalid ones.

The principal challenge in minimal DFA construction is satisfying completeness while using the fewest possible states. For finite languages, a critical structural constraint applies: no productive state may contain a self-loop or participate in a cycle. Any such cycle would generate infinitely many accepted strings, contradicting finiteness.

The construction strategy follows a disciplined sequence:

1. Build transitions for valid strings only.
2. Ignore the full alphabet initially.
3. Complete missing transitions by introducing a single dead state.
4. Ensure no productive cycles exist.

Fundamental Principles of DFA Construction

A DFA must satisfy the following properties:

Total Transition Function

For every state and every input symbol in Σ , a transition must be defined.

Acceptance and Rejection

The automaton must accept every string in the language and reject every string not in the language.

Empty String Acceptance

A DFA accepts ϵ if and only if the initial state is also a final state.

Minimal State Objective

The goal is to represent the language using the smallest number of states while preserving completeness.

Methodology for Constructing Minimal DFAs

Step 1: Process Valid Inputs

Construct states and transitions only for strings belonging to the language.

Step 2: Ignore Full Alphabet Initially

Focus strictly on valid paths. Do not prematurely introduce transitions for unused symbols.

Step 3: Complete the System
After valid paths are built, check every state. Any undefined transition must go to a dead state.

Step 4: Dead State Utilization
A dead state is a non-final state where all outgoing transitions loop to itself. Only one dead state is required in a minimal DFA.

Structural Constraints for Finite Languages

- To preserve finiteness:
 - No Productive Self-Loops
No self-loop may exist on any state that can reach a final state.
 - No Productive Cycles
There must be no cycle among productive states.

Dead-State Loops Allowed
Self-loops are permitted only on the dead state, since it cannot reach a final state.

Detailed Case Studies and State Analysis

The following table summarizes minimal state requirements for selected finite languages.

Language (L)	Alphabet (Σ)	States in mDFA	Structural Note
{a}	{a}	3	Initial $\rightarrow$ Final $\rightarrow$ Dead
{ ϵ , a}	{a}	3	Initial is Final; Dead handles length > 1
{ ϵ , a}	{a, b}	3	'b' transitions go to Dead
{ ϵ , aa}	{a}	4	Separate state for 'a' before 'aa'
{a, ab, ba}	{a, b}	5	Branching from initial state
{aba, ab, bab, ba}	{a, b}	6	Shared prefix paths minimize states
{ab, ba, abba, baba, ... finite set}	{a, b}	7	Careful suffix merging required

$\{w \mid |w| \leq 2\}$

Analysis of Specific Examples

Example 1: $L = \{a\}$

To accept only "a":

State 0 (Initial): on 'a' $\rightarrow$ State 1

State 1 (Final): on 'a' $\rightarrow$ State 2

State 2 (Dead): loops on 'a'

Why 3 states are required:

- One state cannot distinguish ϵ from "a".
- Two states cannot maintain completeness without creating cycles.
- Three states ensure correct acceptance and rejection.

Example 2: $L = \{a, ab, ba\}$

Construction Strategy:

From Initial State:

- On 'a' $\rightarrow$ State A (Final for "a")
- On 'b' $\rightarrow$ State B

From State A:

- On 'b' $\rightarrow$ State AB (Final for "ab")
- On 'a' $\rightarrow$ Dead

From State B:

- On 'a' $\rightarrow$ State BA (Final for "ba")
- On 'b' $\rightarrow$ Dead

All undefined transitions go to Dead.

This branching structure ensures completeness without cycles.

Example 3: Length Constraint $|w| \leq 2$

States represent string length:

State 0: Length 0 (Final)

State 1: Length 1 (Final)

State 2: Length 2 (Final)

State 3: Dead (Length ≥ 3)

All transitions from State 2 lead to Dead.

Critical Insights on Minimization

Avoid Redundant States

If language contains AB and ABA, the state after reading AB should be marked final and extended to accept ABA. Do not duplicate prefix paths.

Path Sharing

Minimal DFAs share common prefixes among strings.

Example:

AB is prefix of ABA.

Reuse states instead of restarting from initial state.

The Fundamental Construction Error

Creating a transition from a final state back to an earlier productive state creates a cycle. This generates infinitely many strings and violates finiteness.

Handling Equal Counts with Length Bound

Example:

Number of a's equals number of b's with total length 4.

The language is finite because maximum length is fixed.

The DFA must:

- Track count difference between a and b.
- Track total length.
- Send any string exceeding length 4 to Dead.
- Send any prefix that cannot possibly achieve equal counts within 4 symbols to Dead.